

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА - Российский технологический университет»
(РТУ МИРЭА)

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

КТ1 Исследовательский этап

Тема

Сборка и тестирование серверного оборудования

Выполнил: Новожилов Василий Алексеевич

Группа: ИКБО-10-24

Место практики: ООО «Бюджетные и Финансовые Технологии»

Руководитель от университета: Синицын Анатолий Васильевич

Руководитель от предприятия: Жиркова М. И.

Москва 2026

Содержание

Введение	3
1 Предметная область	4
2 Описание основного бизнес-процесса	7
3 Среда разработки и система контроля версий	11
4 Перевод фрагмента иностранного источника	14
Заключение	17
Список использованных источников	18

Введение

Тема учебной практики - «Сборка и тестирование серверного оборудования». Место практики - ООО «Бюджетные и Финансовые Технологии». В работе предлагается организация учёта сборки и приёмочных испытаний сервера.

После сборки необходимо проверить оборудование и сохранить сведения о комплектации, результатах и выявленных несоответствиях. Раздельное хранение этих данных затрудняет подготовку протокола. Поэтому предлагается объединить их вокруг заявки и экземпляра оборудования.

Объект исследования - подготовка серверного оборудования к эксплуатации. Предмет исследования - учёт комплектации, результатов испытаний и решения о готовности сервера к передаче. Границы процесса: от получения утверждённой спецификации и комплекта деталей до оформления протокола приёмки.

Цель практики - разработать программное средство учёта сборки и тестирования серверов, которое сохраняет результаты проверок и выявляет неполноту протокола.

Задачи соответствуют этапам практики. На исследовательском этапе необходимо изучить область, описать процесс, собрать источники, выполнить перевод и подготовить среду разработки. На этапе моделирования - описать процесс в требуемой нотации и определить данные. На этапе разработки - реализовать учёт заказов, комплектации и испытаний. На этапе проверки - испытать программу, исправить замечания и подготовить итоговую документацию.

Используются анализ публикаций и документации, текстовое описание процесса и проверка прототипа. Требования к системе уточняются с учётом ГОСТ 34.602-2020 [6]. Описанный порядок является проектным предложением: фактические показатели работы предприятия и достигнутая экономия не заявляются. Примеры данных являются учебными.

1 Предметная область

1.1 Состав и развитие предметной области

Серверное оборудование обеспечивает выполнение вычислительных задач, хранение данных и предоставление сетевых сервисов. В рамках темы практики рассматривается отдельный сервер как изделие с определённой конфигурацией: процессорами, оперативной памятью, накопителями, сетевыми интерфейсами, блоками питания и системой охлаждения. При сборке требуется согласовать эти элементы между собой и с назначением изделия.

Изменение рынка влияет на условия комплектования серверов. В исследовании В. Е. Забегаевой и С. Д. Володина анализируется российский рынок на данных о продажах за 2018-2020 годы и рассматриваются изменения условий поставок после 2022 года [1]. Эта работа используется для характеристики развития предметной области. Её результаты не следует считать статистикой состояния рынка на октябрь 2026 года. Для проектируемой системы существенна необходимость фиксировать фактический состав оборудования и согласованные замены.

Одно из направлений развития диагностики - переход от отдельных измерений к накоплению и совместному анализу данных. В статье M. Zhang и соавторов описано применение технологии S.M.A.R.T., сформировавшейся в 1990-е годы, и исследованы методы прогнозирования отказов накопителей на основе её показателей [10]. Для рассматриваемого проекта это обосновывает сохранение диагностической информации, а не только итоговой отметки «успешно».

На современном этапе исследуются также сочетания разных групп системных метрик. Т. R. Chhetri и соавторы объединяют показатели S.M.A.R.T. с другими метриками серверов и сравнивают методы прогнозирования отказов [9]. Эти исследования показывают направление развития средств диагностики, однако учебный проект не предполагает создание собственной модели машинного обучения. Сначала необходимо обеспечить последовательный сбор достоверных результатов.

Развитие централизованного журналирования показано в работе А. Г. Абрамова [2]. Автор рассматривает сбор и анализ журналов сетевого и серверного оборудования в национальной исследовательской компьютерной сети. Применительно к учебному проекту из этого следует проектное решение: журналы должны быть связаны с конкретным сервером и конкретным прогоном испытаний, чтобы инженер мог восстановить обстоятельства ошибки.

1.2 Особенности сборки и испытаний

Сборка начинается с проверки совместимости и комплектности. Совпадение названия модели само по себе не описывает всю конфигурацию: существенны количество модулей, объём памяти, состав накопителей, версия встроенного программного обеспечения и расположение компонентов. В карточке изделия следует различать плановую и фактическую комплектацию. Замена детали должна приводить к обновлению соответствующей записи и оценке необходимости повторных испытаний.

Испытания и контроль качества имеют установленную терминологию, приведённую в ГОСТ 16504-81 [5]. В данном проекте контроль комплектации отвечает на вопрос о соответствии фактического состава спецификации, а испытания проверяют поведение собранного изделия в заданных условиях. Успешная загрузка операционной системы не заменяет весь согласованный набор проверок.

План испытаний должен соответствовать назначению сервера. Для вычислительного узла существенны проверки процессора и памяти, для сервера хранения - параметры накопителей и подсистемы ввода-вывода. Общая структура проекта предусматривает проверки процессора, памяти, накопителей, сети, охлаждения и питания. Конкретные утилиты, длительность, нагрузка и допустимые значения выбираются для модели оборудования и согласуются перед проведением испытаний.

Работа О. С. Аладышева и соавторов посвящена выбору параллельной файловой системы и ожиданиям от производительности вычислительного кластера [3]. Она рассматривает более крупную систему, чем один собираемый сервер. Для данной работы важен принцип соответствия

проверки предполагаемой нагрузке: результат одного теста нельзя без дополнительного обоснования переносить на любое применение оборудования.

Нужно различать аппаратное несоответствие, ошибку диагностического средства и ошибку оформления результата. А. Н. Привалов, Е. В. Ларкин и А. В. Богомоллов рассматривают надёжность программных компонентов и отмечают зависимость результатов тестирования от его длительности и покрытия входных данных [4]. В учебном проекте это учитывается при проверке собственного обработчика протоколов: отдельно проверяются неполные данные, неизвестные значения и ошибочные результаты.

Форму программы и методики испытаний разрабатываемого программного средства можно строить с учётом ГОСТ 19.301-79 [7]. Этот стандарт относится к программной документации; он не подменяет инструкции изготовителя по аппаратным испытаниям. Состав эксплуатационных сведений о самом изделии целесообразно согласовывать с подходом ГОСТ Р 2.610-2019 к паспортам, формулярам и другим эксплуатационным документам [8].

1.3 Обоснование автоматизации

Автоматизируется движение информации о сборке и проверках. Программа должна уменьшать вероятность потери данных и помогать соблюдать порядок работ; физическая сборка и инженерная ответственность остаются за специалистами.

При ручном учёте возможны повторный ввод серийных номеров, путаница версий спецификации и пропуски проверок. Перезапись неудачного результата уничтожает историю исправлений, а отсутствие связи журнала с заявкой затрудняет поиск. Частоту этих рисков в организации необходимо установить отдельно.

Система связывает заявку, конфигурацию, план и результаты. Отсутствие результата не считается успехом. Повторная проверка создаёт новый прогон с сохранением предыдущего. Итоговый протокол передаётся ответственному за приёмку.

2 Описание основного бизнес-процесса

2.1 Границы и участники

Выбранный процесс - «Сборка и приёмочное тестирование сервера по утверждённой спецификации». Начальное событие - передача инженеру согласованной заявки, спецификации и доступного комплекта деталей. Конечное событие - оформление решения о готовности сервера к передаче либо возврат изделия на доработку с зафиксированными несоответствиями.

Ответственным за прохождение заявки назначается инженер, координирующий сборку и испытания. Инициатор заявки определяет назначение сервера и требования к результату. Ответственный за комплектование выдаёт детали и документы. Инженер сборки проверяет состав и выполняет сборку. Инженер испытаний запускает проверки и фиксирует результаты. Ответственный за приёмку рассматривает протокол и принимает решение о передаче. Одно лицо может выполнять несколько ролей, но в записях должно быть понятно, кто совершил каждое действие.

В процесс не входят закупки, разработка электронных плат, серийное производство компонентов и обслуживание после передачи. Развёртывание прикладных систем заказчика также рассматривается отдельно. Такой масштаб позволяет назвать единый вход и результат, а затем моделировать и автоматизировать процесс на следующих этапах практики.

2.2 Входные данные

Входом являются заявка с уникальным номером, назначение сервера и утверждённая версия спецификации. Спецификация задаёт состав оборудования и параметры, с которыми будет сравниваться фактическая конфигурация. От ответственного за комплектование поступают детали, их идентификаторы и сопроводительная документация.

Дополнительно требуется план испытаний. Для каждой обязательной проверки в нём определяются объект, средство выполнения, условия, ожидаемый результат и критерий оценки. Числовые ограничения нельзя назначать одинаковыми для всех серверов: они должны соответствовать документации изготовителя и согласованным требованиям. Версия плана

сохраняется вместе с результатами, чтобы изменение методики не меняло смысл старых протоколов.

Если заявка, спецификация или необходимые детали отсутствуют, процесс не должен переходить к оформлению готового изделия. Несоответствие фиксируется, после чего ответственный запрашивает уточнение или замену. Согласованная замена отражается в новой версии спецификации.

2.3 Последовательность выполнения

Первый шаг - регистрация заявки и назначение ответственного. В карточке фиксируются номер, назначение сервера, версия спецификации и срок выполнения. При повторном обращении по тому же изделию инженер должен видеть существующую запись, а не создавать несвязанную копию.

Второй шаг - входная проверка компонентов. Инженер сверяет модели, количество, маркировку и совместимость с документацией. При нехватке детали или несовпадении параметров работа приостанавливается до согласования. Выявленное несоответствие записывается в карточку заявки.

Третий шаг - сборка. Компоненты устанавливаются по руководствам изготовителей. Проверяются соединения, крепления и организация охлаждения. Фактическая комплектация и серийные номера заносятся в карточку оборудования. Правила безопасной работы определяются инструкциями предприятия и изготовителя, а не программой учёта.

Четвёртый шаг - первое включение и первичная проверка. Инженер контролирует прохождение POST, доступность средств управления и обнаружение установленных компонентов. Фиксируются версии BIOS/UEFI и BMC, а также параметры, способные повлиять на результаты испытаний. Неуспешное включение направляет изделие на поиск причины и доработку.

Пятый шаг - сверка конфигурации. Обнаруженные процессоры, память, накопители и сетевые интерфейсы сравниваются с утверждённой спецификацией. Несоответствие не допускается скрывать общей отметкой об успешной загрузке. После устранения причины сверка выполняется повторно.

Шестой шаг - выполнение согласованного плана испытаний. Для каждой проверки сохраняются время, исполнитель, условия запуска, результат и исходный журнал либо ссылка на него. В первоначальном прототипе применяются состояния pass, fail и pending. Отсутствующая запись считается пропущенной проверкой. Проверки, меняющие содержимое накопителей, выполняются только на выделенном для испытаний оборудовании после согласования.

Седьмой шаг - обработка обнаруженного дефекта. Инженер описывает проявление, связывает его с прогоном и фиксирует выполненное исправление. После замены компонента или изменения настроек создаётся новый прогон испытаний. Набор повторных проверок определяется характером изменения и планом испытаний. Первоначальный результат сохраняется.

Восьмой шаг - подготовка протокола. В него включаются номер заявки, идентификатор сервера, фактическая конфигурация, версия плана, результаты обязательных проверок, сведения о дефектах и исполнитель. Программа проверяет полноту записей и формирует предварительный признак готовности.

Девятый шаг - приёмка. Ответственный рассматривает протокол и подтверждающие материалы. Положительное решение возможно при соответствии конфигурации, успешном завершении всех обязательных проверок и отсутствии открытых дефектов. После решения сервер и документы передаются ответственному за эксплуатацию. При отрицательном решении формируется перечень доработок.

2.4 Выход и исключительные ситуации

Положительный выход процесса - собранный сервер, комплект сведений о нём и протокол с решением о готовности к передаче. Получателем является ответственный за эксплуатацию либо иной назначенный представитель инициатора. Отрицательный выход - документированный перечень несоответствий и возврат изделия на доработку.

Пропущенный тест, прерванный запуск и неизвестный результат не приравниваются к успешному прохождению. При отсутствии журнала

инженер должен выяснить, можно ли подтвердить результат другим предусмотренным способом, или повторить проверку. Неверный идентификатор изделия требует исправления связи с заявкой до приёмки.

Программный признак готовности является вспомогательным. Он показывает выполнение формальных условий, но не подтверждает достоверность введённых сведений и не заменяет решение ответственного специалиста. В частности, запись pass без фактически проведённого испытания не доказывает исправность оборудования.

2.5 Узкие места и требования к системе

Первое узкое место - повторное переписывание комплектации. Для его сокращения данные вводятся в карточку один раз и используются в последующих документах. При изменении состава необходима история изменений, чтобы новый перечень не подменял сведения о ранее выполненном прогоне.

Второе узкое место - поиск результатов среди отдельных файлов. Для устранения этой проблемы каждый прогон должен иметь собственный идентификатор и связь с заявкой, сервером, исполнителем и версией плана. Журналы привязываются к конкретным проверкам.

Третье узкое место - оформление протокола при неполном наборе результатов. Система должна проверять обязательные поля, различать неуспех и отсутствие данных и запрещать положительный итог при открытых дефектах. Для приёмки требуется понятный список причин, а не только булево значение.

На КТ1 эти требования проверены в небольшом обработчике учебного JSON-протокола. На следующих этапах необходимо добавить хранение записей в PostgreSQL, роли пользователей, историю изменений и формирование полного протокола. Решения о подключении конкретных диагностических утилит принимаются после согласования оборудования и плана испытаний.

3 Среда разработки и система контроля версий

3.1 Язык и редактор

Для разработки выбран Dart SDK 3.13.5. Язык подходит для приложений командной строки и серверных программ; эти варианты использования представлены в официальной документации [11]. Стандартные библиотеки позволяют читать файлы, разбирать JSON и взаимодействовать с операционной системой. Начальный прототип выполняется в командной строке и не требует Flutter.

Dart выбран как основной язык учебного проекта, чтобы использовать статическую проверку типов и единый набор инструментов форматирования и анализа. По сравнению с Python он требует более явного описания типов при развитии проекта. По сравнению с Rust он позволяет сосредоточиться на обработке данных без изучения модели владения памятью. Эти особенности рассматриваются как аргументы выбора для данной учебной задачи, а не как утверждение о превосходстве языка во всех случаях.

В качестве редактора выбран Visual Studio Code с поддержкой Dart. Редактор позволяет работать с исходным кодом, терминалом и Git в одном окне; официальные материалы выпуска приведены в источнике [14]. Применение отдельной большой IDE для начального консольного проекта не требуется. Проверки выполняются командами SDK, поэтому не зависят от конкретного редактора.

На KT1 используются команды `dart format` для форматирования, `dart analyze` для статического анализа и `dart run` для запуска. Код размещён в `bin/check.dart`, проверка поведения - в `test/check_test.dart`. Внешние пакеты к начальному прототипу не подключены, зависимости окружения ограничены SDK.

3.2 Хранение данных

Для хранения заявок и результатов выбрана PostgreSQL 16. Локальная проверка начальной схемы выполнена на PostgreSQL 16.13. Документация семейства PostgreSQL 16 приведена в источнике [12]. Реляционная структура позволяет связывать заявку, её компоненты и результаты испытаний внешними ключами, а ограничения базы помогают не допускать пустых обязательных значений и повторяющихся идентификаторов.

По сравнению с набором JSON-файлов база данных удобнее для совместной работы и поиска связанных записей. SQLite могла бы использоваться в однопользовательском прототипе, однако дальнейшая система предполагает несколько участников процесса. Поэтому выбрана серверная СУБД. Развёртывание кластера и настройка сложной репликации на данном этапе не требуются.

Начальная схема содержит таблицы `orders`, `components` и `test_runs`. Первая хранит заявку, серийный номер сервера и спецификацию; вторая - сведения о компонентах; третья - сведения о прогонах испытаний. Для переменного состава спецификации и результатов предусмотрен JSONB. Проверка обязательных групп испытаний пока выполняется в Dart, а SQL-схема задаёт связи и базовые ограничения.

На КТ1 консольный обработчик ещё не обращается к базе. Для дальнейшего подключения PostgreSQL из Dart рассматривается пакет `postgres` [15]. Его добавление имеет смысл одновременно с реализацией сохранения и чтения заявок. Такой порядок позволяет сначала проверить правила обработки данных без дополнительных зависимостей.

3.3 Репозиторий и порядок работы

Для контроля версий используется Git 2.52.0. В репозитории сохраняются исходный код, схема БД, описание процесса и материалы отчёта. Git подходит для локальной работы и последующей публикации на GitHub. Сообщения коммитов отражают конкретные изменения, чтобы по истории можно было определить назначение сохранённого состояния.

В корне проекта находятся README.md с темой, ФИО и командами запуска, а также .gitignore. Во временную папку вынесены SDK, окружение подготовки отчёта, промежуточные файлы и тестовая база. Они не включаются в историю проекта. Учебный пример протокола хранится отдельно от результатов реальных испытаний.

Выгрузка истории строится средствами `git log` с параметрами `--all`, `--date=iso` и `--pretty=format:"%h %ad %an %s"` [13]. Файл `git_log.txt` включается в каталог `vcs` сдаваемого архива. Адрес удалённого репозитория должен быть записан одной строкой в `hero_url.txt` после создания репозитория на GitHub.

Изменения проверяются локально и сохраняются отдельными коммитами. Выгрузка истории обновляется перед сборкой сдаваемого архива.

3.4 Проверка начального прототипа

Вход обработчика - JSON-объект с номером заявки, серийным номером сервера, версией плана, признаком соответствия конфигурации, числом открытых дефектов и результатами шести обязательных групп проверок. Пустые идентификаторы, отрицательное число дефектов и неизвестные значения результатов отклоняются как неверные входные данные.

В результате запуска формируются идентификаторы протокола, признак `ready` и список причин, препятствующих допуску. Успешный пример возвращает `ready=true`. Неуспешная, ожидающая выполнения или пропущенная проверка приводит к `ready=false`. Несоответствие комплектации и открытые дефекты также блокируют положительный итог.

Для проверки подготовлен исполняемый сценарий без отдельного тестового фреймворка. Он охватывает успешный протокол, отказ и пропуск проверки, незавершённый тест, несовпадение конфигурации, наличие дефекта и некорректные входные данные. Сценарий выполнен успешно; статический анализ Dart замечаний не выявил. SQL-схема создана в отдельной временной базе PostgreSQL, после чего тестовый сервер остановлен.

Проверены только перечисленные сценарии учебной программы. Испытания реального оборудования и всей системы относятся к следующим этапам.

4 Перевод фрагмента иностранного источника

4.1 Сведения об оригинале

Переведены аннотация и раздел 5 Conclusions статьи T. R. Chhetri, C. K. Dehury, A. Lind, S. N. Srirama и A. Fensel «A Combined System Metrics Approach to Cloud Service Reliability Using Artificial Intelligence» [9]. Статья опубликована в журнале Big Data and Cognitive Computing в 2022 году, том 6, выпуск 1, статья 26; DOI: 10.3390/bdcc6010026.

Русское название: «Повышение надёжности облачных сервисов с использованием искусственного интеллекта на основе совместного анализа системных метрик». Оригинальные страницы 1 и 24 приложены в translation/original.pdf. Перевод относится только к названным разделам на этих страницах; соседние фрагменты статьи в него не включены.

Статья опубликована по лицензии Creative Commons Attribution 4.0. Правообладатели - авторы статьи, 2022 год. Настоящий русский перевод является адаптацией выбранного фрагмента; английский оригинал сохранён без изменения текста. Условия лицензии: <https://creativecommons.org/licenses/by/4.0/>. Номера ссылок внутри переведённого заключения обозначены как номера в оригинале, чтобы отличать их от списка источников настоящего отчёта.

4.2 Перевод аннотации

Выявление и прогнозирование возможных отказов в облачной среде - эффективный способ повышения её надёжности и предупреждения отказов. Проведено много исследований, посвящённых прогнозированию потенциальных отказов, однако ни в одном из них показатели жёстких дисков S.M.A.R.T. - технологии самоконтроля, анализа и отчётности - не объединялись с другими системными метриками, например загрузкой центрального процессора. Поэтому для повышения надёжности мы предлагаем подход к прогнозированию отказов на основе искусственного интеллекта, использующий совместный анализ системных метрик.

Мы проверили этот подход на данных более чем ста облачных серверов, используя четыре алгоритма искусственного интеллекта: случайный лес,

градиентный бустинг, сеть долгой краткосрочной памяти и управляемый рекуррентный блок. Кроме того, был проведён корреляционный анализ. Он позволил выявить связи между системными метриками и отказами. Экспериментальные результаты показывают преимущества объединения системных метрик и превосходят результаты современных методов, рассмотренных в исследовании.

Ключевые слова: прогнозирование отказов; отказоустойчивость; облачные вычисления; искусственный интеллект; надёжность.

4.3 Перевод заключения

Мы предположили, что объединение нескольких системных метрик улучшит прогнозирование отказов, и в этой статье представили исследование прогнозирования отказов облачных серверов. Мы также обосновали эту гипотезу и показали преимущества объединённых метрик с помощью анализа данных, исследования корреляций и экспериментальных результатов, полученных на собранных нами данных ста различных облачных серверов.

В результате анализа данных и исследования корреляций мы получили следующие наблюдения. Во-первых, системные показатели S.M.A.R.T. очень слабо коррелируют с другими метриками, например загрузкой процессора, либо практически не имеют с ними корреляции. Во-вторых, при отказе может наблюдаться высокая загрузка ресурсов. В-третьих, объединение системных метрик даёт преимущества. Например, показатели S.M.A.R.T. не включают информацию об отказах, тогда как другие системные метрики её содержат, что рассмотрено в разделе 4.4.1 исходной статьи.

Далее при сравнении наших моделей автоматического прогнозирования отказов мы обнаружили, что все четыре алгоритма искусственного интеллекта - случайный лес, градиентный бустинг, сеть долгой краткосрочной памяти и управляемый рекуррентный блок, - использующие объединённые системные метрики, то есть мультимодальные входные данные, превзошли результаты рассмотренных современных исследований. При этом наибольшую долю правильных прогнозов по сравнению с альтернативами обеспечили методы случайного леса.

Кроме того, по причинам конкуренции облачные провайдеры и центры высокопроизводительных вычислений не заинтересованы в публикации сведений о своих отказах. Это также объясняет, почему наборы данных об отказах редки, а существующие наборы сильно устарели. Использование таких старых данных не даёт необходимого представления о текущем состоянии, в частности из-за развития аппаратного обеспечения и технологий.

Поэтому нам пришлось сформировать целевую метку; аналогичная процедура применяется и в других исследованиях и описана в разделе 3.3 исходной статьи. Однако такой подход может неточно отражать реальную картину отказов, поскольку иногда они возникают внезапно. Мы рассматриваем это как ограничение исследования.

Недавняя работа Chhetri и соавторов о прогнозировании отказов жёстких дисков с применением графов знаний продемонстрировала ещё более высокую эффективность. В оригинале здесь приведены ссылки 100 на эту работу и 101 на материал о графах знаний. В дальнейшем предполагается развивать подход, объединяющий системные метрики на основе графов знаний, описанный в работе под номером 100, чтобы добиться ещё большей эффективности, а также реализовать метод в производственной среде.

4.4 Связь публикации с темой практики

Для проекта полезна идея сохранять разные группы диагностических данных вместе. Вместе с тем статья посвящена прогнозированию отказов в облачной инфраструктуре, а наш процесс - приёме отдельного собранного сервера. Поэтому показатели качества моделей из статьи не переносятся на учебный прототип. Формальные признаки pass и fail также не являются аналогом прогнозной модели. На текущем этапе используются проверяемые правила полноты протокола; возможность дальнейшего анализа данных зависит от накопления реальных результатов.

Заключение

На исследовательском этапе выбран процесс сборки и приёмочного тестирования сервера по утверждённой спецификации. Определены его границы, участники, входные данные, последовательность действий и итоговый документ. Сформулированы требования к учёту комплектации, хранению результатов и проверке полноты протокола.

Для разработки выбраны Dart и PostgreSQL. Подготовлены локальный репозиторий, начальная схема данных и консольный прототип проверки учебного протокола. Выполнены проверки основных положительных и отрицательных сценариев. Собрана библиография и переведён фрагмент иностранной публикации по теме надёжности серверов.

Следующий этап - моделирование того же процесса и уточнение структуры данных и плана испытаний. Затем необходимо реализовать хранение заявок и результатов, пользовательские роли и формирование полного протокола. Переход к реальным испытаниям требует согласованного набора оборудования и критериев оценки.

Список использованных источников

1. Забегаева, В. Е. Рынок серверного оборудования в России: анализ конкуренции до событий 2022 года / В. Е. Забегаева, С. Д. Володин // Научные исследования экономического факультета. Электронный журнал. - 2023. - Т. 15, № 1. - С. 109-125. - DOI 10.38050/2078-3809-2023-15-1-109-125. - Текст : электронный. - URL: https://archive.econ.msu.ru/journal/issues/2023/2023.volume_15.issue_1/Zabegaeva-Volodin/ (дата обращения: 05.10.2026).
2. Абрамов, А. Г. Высокопроизводительный сервис сбора и анализа файлов журналов сетевого и серверного оборудования в национальной исследовательской компьютерной сети / А. Г. Абрамов // Программные продукты и системы. - 2024. - Т. 37, № 4. - С. 495-503. - DOI 10.15827/0236-235X.148.495-503. - Текст : электронный. - URL: <https://swsys.ru/index.php?id=5113&page=article> (дата обращения: 05.10.2026).
3. Ожидания от производительности вычислительного кластера при выборе параллельной файловой системы / О. С. Аладышев, А. В. Захарченко, В. Ф. Огарышев, Б. М. Шабанов // Программные продукты и системы. - 2024. - Т. 37, № 4. - С. 472-486. - DOI 10.15827/0236-235X.148.472-486. - Текст : электронный. - URL: <https://doi.org/10.15827/0236-235X.148.472-486> (дата обращения: 05.10.2026).
4. Привалов, А. Н. Моделирование надежности программных компонентов киберфизических систем / А. Н. Привалов, Е. В. Ларкин, А. В. Богомолов // Программные продукты и системы. - 2025. - Т. 38, № 1. - С. 47-54. - DOI 10.15827/0236-235X.149.047-054. - Текст : электронный. - URL: <https://swsys.ru/index.php?page=article&id=5134> (дата обращения: 05.10.2026).
5. ГОСТ 16504-81. Система государственных испытаний продукции. Испытания и контроль качества продукции. Основные термины и определения : дата введения 1982-01-01. - Переиздание с изменением № 1. - Москва : Стандартинформ, 2011. - 25 с. - Текст : электронный. - URL: <https://www.gostinfo.ru/catalog/Details/?id=4118306> (дата обращения: 05.10.2026).

6. ГОСТ 34.602-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы : дата введения 2022-01-01. - Издание с поправками. - Москва : Российский институт стандартизации, 2026. - 12 с. - Текст : электронный. - URL: <https://www.gostinfo.ru/catalog/Details/?id=6877429> (дата обращения: 05.10.2026).
7. ГОСТ 19.301-79. Единая система программной документации. Программа и методика испытаний. Требования к содержанию и оформлению : дата введения 1981-01-01. - Переиздание с изменениями № 1, 2. - Москва : Стандартинформ, 2010. - 3 с. - Текст : электронный. - URL: <https://www.gostinfo.ru/catalog/Details/?id=8147133> (дата обращения: 05.10.2026).
8. ГОСТ Р 2.610-2019. Единая система конструкторской документации. Правила выполнения эксплуатационных документов : дата введения 2020-02-01. - Переиздание. - Москва : Стандартинформ, 2021. - 50 с. - Текст : электронный. - URL: <https://www.gostinfo.ru/catalog/Details/?id=8137354> (дата обращения: 05.10.2026).
9. A Combined System Metrics Approach to Cloud Service Reliability Using Artificial Intelligence / T. R. Chhetri, C. K. Dehury, A. Lind [et al.] // Big Data and Cognitive Computing. - 2022. - Vol. 6, no. 1. - Art. 26. - DOI 10.3390/bdcc6010026. - Текст : электронный. - URL: <https://www.mdpi.com/2504-2289/6/1/26> (дата обращения: 05.10.2026).
10. Hard Disk Failure Prediction Based on Blending Ensemble Learning / M. Zhang, W. Ge, R. Tang, P. Liu // Applied Sciences. - 2023. - Vol. 13, no. 5. - Art. 3288. - DOI 10.3390/app13053288. - Текст : электронный. - URL: <https://www.mdpi.com/2076-3417/13/5/3288> (дата обращения: 05.10.2026).
11. Dart. Command-line and server apps / Google. - 2026. - Текст : электронный. - URL: <https://dart.dev/server> (дата обращения: 05.10.2026).

12. PostgreSQL 16 Documentation / The PostgreSQL Global Development Group. - 2026. - Текст : электронный. - URL: <https://www.postgresql.org/docs/16/index.html> (дата обращения: 05.10.2026).
13. Git. git-log Documentation : version 2.52.0 / The Git Project. - 2025. - Текст : электронный. - URL: <https://git-scm.com/docs/git-log/2.52.0> (дата обращения: 05.10.2026).
14. Visual Studio Code. August 2025 : version 1.104 / Microsoft. - 2025. - Текст : электронный. - URL: https://code.visualstudio.com/updates/v1_104 (дата обращения: 05.10.2026).
15. postgres : PostgreSQL client for Dart : version 3.5.19 / Agilord. - 2026. - Текст : электронный. - URL: <https://pub.dev/packages/postgres/versions/3.5.19> (дата обращения: 05.10.2026).